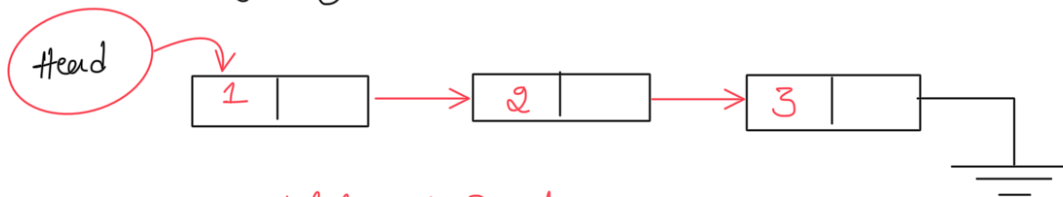


Delete & Search In Singly LinkedList

17/08/25 DAY 38

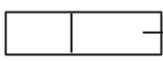
① Delete At Beginning.



perform delete At front

case ① if Head == null can't delete return

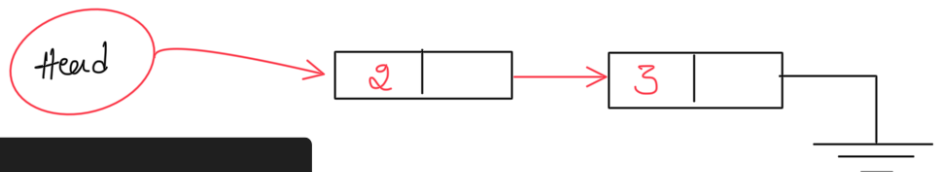
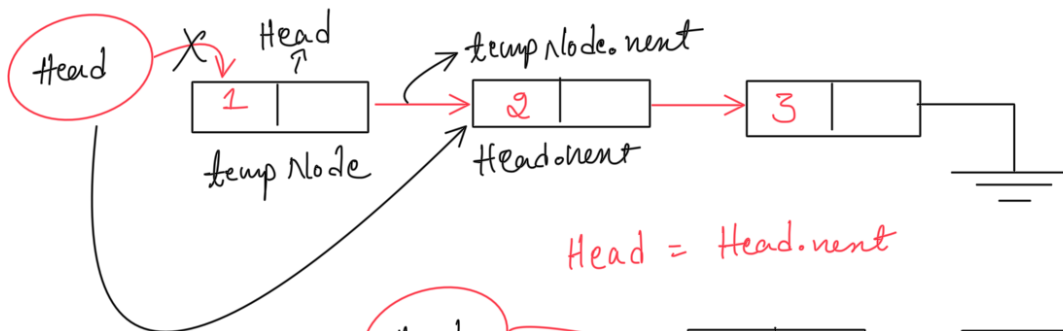
If List Have Single Node

case ② if Head.next == null & Head != null
Head  Null
assign Head to null
& decrement size

case ③ take tempNode assign Head Node Address

tempNode = Head
Head = tempNode.next.

or Head = Head.next
& decrement size



optional

```
108  
109 public void deleteAtFront() {  
110  
111     if (this.head == null) {  
112         System.out.println("List Is Empty ");  
113         return;  
114     }  
115  
116     if (this.head.next == null) {  
117         this.head = null;  
118         this.size--; //decrement size  
119         return;  
120     }  
121  
122     this.head = this.head.next;  
123     this.size--; //decrement size  
124  
125  
126 }
```

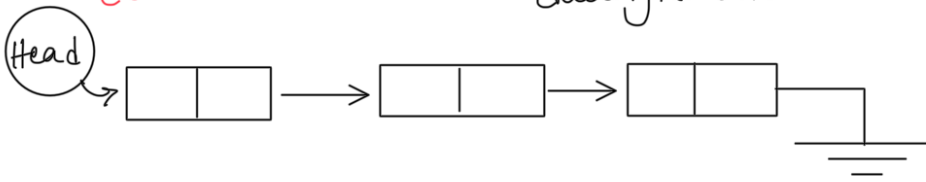
→ If List is Empty don't perform Any delete Operation Bcz Head is already Null

→ If List Have single Node assign null value to Head

→ update Head & decrement size

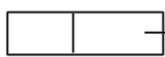
delete Node At End

Deleting Node At the End

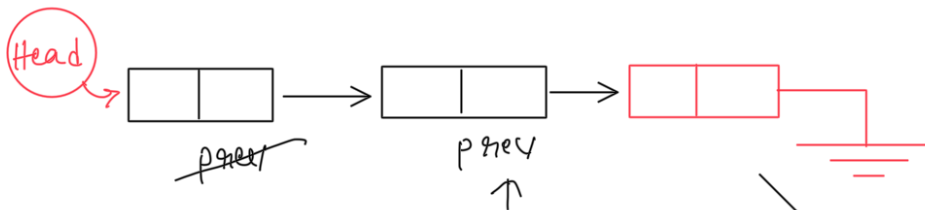


case ① if $Head == null$ can't delete return

If List Have Single Node

case ② if $Head.next == null$ $\Rightarrow$ $Head = null$
 Head  $\Rightarrow$ Null
 assign Head to null
 & decrement size

case ③ travel till last previous Node
 update last previous Node's next as Null

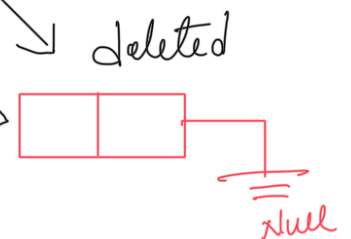
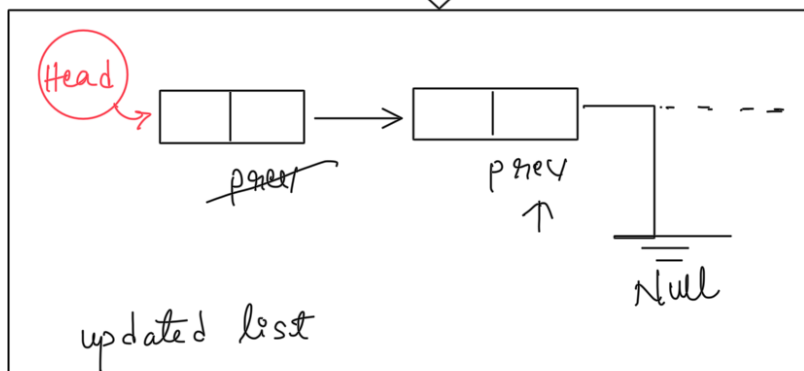


Run Loop untill
 $prev.next.next \neq null$

update

$prev.next = null$

& decrement size



Run Loop untill
 $prev.next.next \neq null$
 $prev = prev.next$;

which points to last
 Second Node

```

126
127 public void deleteAtEnd(){
128
129     if(this.head==null){
130         System.out.println(x:"List Is Empty ");
131         return;
132     }
133
134     if(this.head.next==null){
135         this.head=null;
136         this.size--; //decrement size
137         return;
138     }
139
140     Node prev = head;
141
142     while(prev.next.next != null){
143         prev = prev.next;
144     }
145
146     prev.next = null;
147
148     this.size--; //decrement size
149 }

```

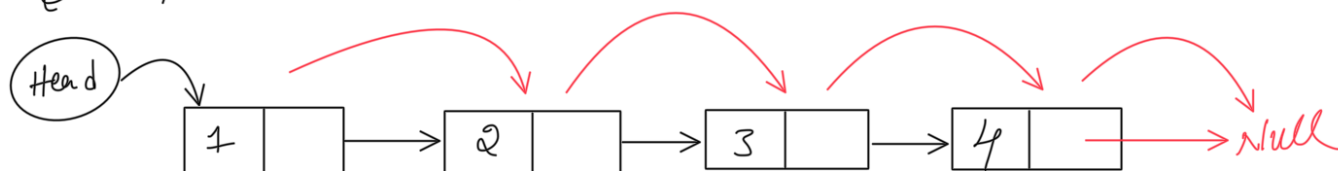
→ If List is Empty dont perform Any delete Operation Bcz Head is Already Null

→ If List Have single Node assign null value to Head decrement size

→ Travel till Last Second Node
update last prev Node next to null.

→ update size of List

Search In A Linked List



Take a pointer
Traverse from Head to Null

Case 1 if Head == Null
list is Empty So key is Not present. return false

Case 2 if list is Not Null

Loop till cur != Null
if key found print return

start from Head Initialize cur to Head

if cur.data == key
print key found
return true

update cur = cur.next

if key Not found in list
print key Not found
return false

